2025-2026

Welcome to the 15th Lab session of CO3519 module (Semester 02)

For Two Hours Lab Session:

#	Task	Time (m)	Guidance
A	Please perform the **Task01** given on **Page 2**.	50	This task is to understand how the transfer learning works and how it is applied for Facial Expression Recognition. We consider VGG16 as it is one of the popular CNN architectures. This task starts on **Page 2** to **Page 7**
	Break	10	
B	Please perform the **Task02** given on **Page 7**	50	In this task, please work on your literature review section of your assignment which is the review of Facial Expression Recognition methods using deep learning. Completing this task will help you build understanding how FER works and also a step towards finishing one section of your assignment.

For One Hour Lab Session:

#	Task	Time (m)	Guidance
1	Please perform the Task **A** from above two hours lab session with the given instructions.	50	Follow the related guide in Task **A**.

I recommend to download this PDF file so that you can view each line of code clearly.

Applying Transfer Learning TL for Facial expression Recognition using VGG16

Important Note: Every student is advised to carefully review each line and develop a clear understanding of the concepts. There is repetition of some concepts covered in lecture in this sheet. This will also support your preparation for the second assessment.

Task01: Facial Expression Recognition using VGG16 by applying Transfer Learning

Last week, we explored the implementation of a custom Convolutional Neural Network (CNN) for facial expression recognition, focusing on classifying six basic emotions. Through this approach, we gained an understanding of how CNNs can be designed from scratch to extract features and make predictions.
This week, we shift our focus to advanced CNN architectures, which consist of deeper networks with multiple layers, each designed to capture intricate patterns in images. These architectures are typically trained on large-scale datasets containing millions of images, making training from scratch computationally expensive and time-intensive. Instead, we leverage transfer learning, a technique that allows us to utilize pre-trained models, such as VGG16, to extract meaningful features from smaller datasets efficiently. By applying transfer learning with VGG16, we aim to enhance model performance while significantly reducing training time and data requirements.
About VGG16: This is a pretrained deep learning model that has been trained on millions of images. We use it because training a CNN from scratch requires a massive dataset and computational power. By using VGG16, we leverage pre-learned features (like edge detection and texture recognition) instead of learning everything from zero.
In the following code, I will skip the description of few pieces of codes as we aware of their functions etc.
First of all, import the essential libraries as follows.

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
from tensorflow.keras.applications import VGG16
import os
```

ImageDataGenerator: This is used for data augmentation. A major issue in deep learning is data scarcity, we often don't have enough images to generalize well. Image augmentation helps by artificially increasing the dataset size through transformations like flipping, rotating, and scaling images.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier("/content/drive/My Drive/haarcascade_frontalface_default.xml")
```

Converting to grayscale: Color is not needed for face detection. Grayscale images are simpler and faster to process because they contain only intensity values (rather than three RGB channels).
Face detection using detectMultiScale()

scaleFactor=1.1: The image is scaled down by 10% at each step, allowing detection of faces at different sizes.
minNeighbors=5: This filters out false positives by ensuring that at least 5 overlapping detections must occur before considering a region as a face.
minSize=(30, 30): Faces smaller than 30x30 pixels are ignored, as they may lack sufficient details for expression recognition.

Resizing to 224x224 pixels: VGG16 expects a fixed input size of 224x224, so we must resize every image accordingly.

```python
def detect_face(image):

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    if len(faces) == 0:
        return None  # No face detected

    # Assuming the first detected face is the main one
    x, y, w, h = faces[0]
    face = image[y:y+h, x:x+w]
    face = cv2.resize(face, (224, 224))  # Resize face to the size expected by VGG16
    return face
```

Real-world facial expressions appear at different angles, lighting conditions, and positions.
Data augmentation artificially expands the dataset and improves generalization.

rescale=1./255 → Normalizes pixel values from [0,255] to [0,1], which makes training more stable.
rotation_range=20 → Randomly rotates images up to ±20° to account for head tilts.
width_shift_range=0.2, height_shift_range=0.2 → Moves images left/right/up/down by 20%, helping the model recognize off-center faces.
horizontal_flip=True → Adds flipped versions of images, improving recognition of symmetrical expressions.

```python
# Define dataset paths
train_dir = '/content/drive/My Drive/CK_dataset/CK_dataset/train'
test_dir = '/content/drive/My Drive/CK_dataset/CK_dataset/test'

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True
)

test_datagen = ImageDataGenerator(rescale=1./255)

# Load the data with ImageDataGenerator
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),  # VGG16 input size
    batch_size=32,
    class_mode="categorical"
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode="categorical"
)
```

```python
# Load the data with ImageDataGenerator (but modify images to detect faces first)
def preprocess_data_generator(directory, batch_size=32, is_test=False):

    image_files = [f for f in os.listdir(directory) if f.endswith('.jpg') or f.endswith('.png')]
    images = []
    labels = []

    for file in image_files:
        image_path = os.path.join(directory, file)
        img = cv2.imread(image_path)

        # Detect face from the image
        face = detect_face(img)

        if face is not None:
            # Resize the face image and normalize it (VGG16 input size is 224x224)
            face = np.expand_dims(face, axis=0)  # Add batch dimension
            face = np.array(face)  # Convert to numpy array

            # Get corresponding label (assuming label is part of filename, modify as needed)
            label = int(file.split("_")[0])  # Modify this part to suit your label format

            images.append(face)
            labels.append(label)

    images = np.array(images)
    labels = np.array(labels)

    # Convert to categorical labels
    labels = keras.utils.to_categorical(labels, num_classes=6)

    return images, labels
```

```python
# Use the custom data generator for training and testing
train_images, train_labels = preprocess_data_generator(train_dir)
test_images, test_labels = preprocess_data_generator(test_dir, is_test=True)
```

As mentioned several times, VGG16 is a deep CNN, consists of convolutional layers that extract features from images, followed by fully connected layers for classification.

Why use it instead of training from scratch?

Feature Reuse: Instead of learning basic features (edges, textures) from scratch, we reuse VGG16's pre-learned features.

Less Data Required: Training a deep CNN from scratch requires millions of images, but using a pretrained model allows good performance even with small datasets.

Computational Efficiency: Training a full CNN from scratch requires high-end GPUs. Transfer learning allows us to get similar performance much faster.

Why do we need a custom classifier?

VGG16 was originally trained to classify 1000 objects (dogs, cats, cars, etc.), but our task is facial expression recognition, which has only 6 classes (e.g., happy, sad, angry, etc.). The original classification layer is removed, and we add a new fully connected layer to classify emotions.

Breakdown of classifier layers:

Flatten() – Converts the feature maps from VGG16 into a 1D vector for the dense layer.
Dense(512, activation='relu') – A fully connected layer with 512 neurons for learning deep patterns.
Dropout(0.5) – Drops 50% of the neurons randomly to reduce overfitting.
Dense(6, activation='softmax') – The final classification layer, with 6 outputs, corresponding to 6 emotions.

```
# Load pre-trained VGG16 model (excluding the top layer)
base_model = VGG16(weights="imagenet", include_top=False, input_shape=(224, 224, 3))

# Freeze convolutional base (don't train these layers)
for layer in base_model.layers:
    layer.trainable = False

# I commented this line which Print the model summary. You can uncomment if you wantt to see the summary
# base_model.summary()

```

```
# Correctly connect new classification layers
x = Flatten()(base_model.output)   # Flatten the output of convolutional base
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
output_layer = Dense(6, activation='softmax')(x)  # Final output layer

# Create the model
model = Model(inputs=base_model.input, outputs=output_layer)

# Compile the model
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.0001),
              loss="categorical_crossentropy",
              metrics=["accuracy"])

# You can uncomment if you want to see the summary.
# model.summary()

```

You can start training here. Here, I have used 10 epochs to perform training. After using 10, please change the number of epochs and check for 20 epochs. Next, in work as exercise, please compute the confusion matrix and then make single image test.

```
# Train the model
history = model.fit(
    train_generator,
    validation_data=test_generator,
    epochs=10,
    steps_per_epoch=len(train_generator),
    validation_steps=len(test_generator)
)

# Evaluate on test dataset
test_loss, test_acc = model.evaluate(test_generator)
print(f"Test Accuracy: {test_acc * 100:.2f}%")
```

You can plot the accuracy and loss as follows:

```python
import matplotlib.pyplot as plt

# Plot training & validation accuracy and loss
plt.figure(figsize=(12, 4))

# Accuracy Plot
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# Loss Plot
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.show()
```

Task02: FER Literature Review section

In this task, please work on your literature review section of your assignment which is the review of Facial Expression Recognition using deep learning/CNN. Completing this task will help you finishing one section of your assignment.